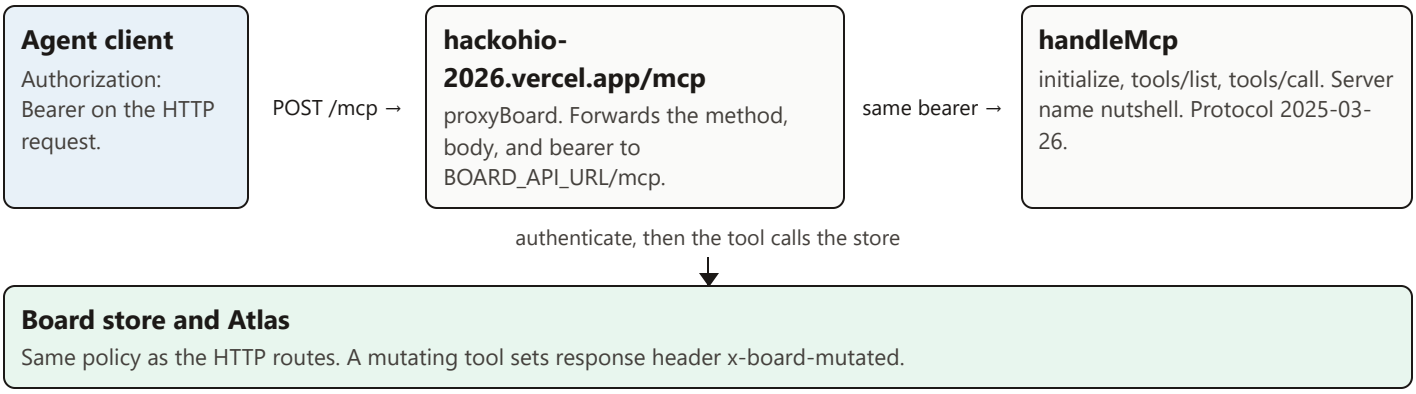
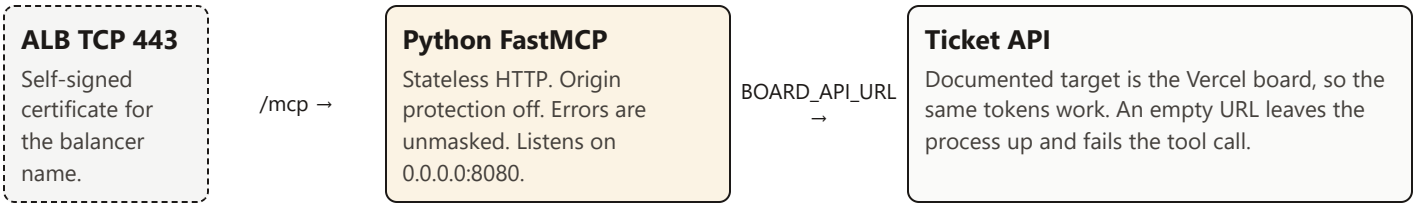
5. Agent data flow

The install URL people copy is the Nutshell MCP route. That route does not run the Python server. It forwards the MCP JSON to the board API, which runs the tools in-process against the same store as the ticket routes.



Tool	Effect	Scope
get_board	GET /tickets. Optional status.	tickets:read
create_ticket	POST /tickets. Starts in backlog.	tickets:write
move_ticket	PATCH /tickets/:key with status and, for an agent, a commit.	tickets:write, and tickets:close to reach done
add_comment	POST /tickets/:key/comments.	comments:write
read_comments	Comments on one ticket. Optional since.	tickets:read
link_commit	PATCH with a commit and no status change.	tickets:write
watch_changes	GET /events, newest first.	tickets:read
list_project_templates	Catalog. Each entry is a board, a table, or a roadmap.	tickets:read
get_project_template	One template, with layout, columns, fields, and starter tags.	tickets:read
get_project_table	One project as groups and rows.	tickets:read
create_project	POST /projects from a template id. Key is the ticket prefix.	tickets:write

The other MCP process



Python tools are get_board, create_ticket, move_ticket, add_comment, link_commit, watch_changes, list_project_templates, get_project_template, create_project, and get_project_table. It does not implement read_comments. The Fargate board task on port 3000 does not hold the Vercel tokens. endpoints.md says not to point this process at that task.

Live events

`GET /events` with `Accept: text/event-stream` stays open. It sends `stream_open`, the stored events, then Redis messages. Channels are `buc2:events`, `buc2:user:<id>`, and for an agent also `buc2:agent:<id>`. Account notices for signup and sign-in go to the user channel.